Week 6 - Pandas

(and a HINT of SQL...)

Data Handling

What are the ways that we have learned so far to handle data?

- Lists of lists
- Dictionaries
- Custom Classes

None of these are particularly conducive to data exploration and quick manipulation

Introducing Data Frames

When we want to manipulate data sets in a clean and efficient manner, we want to start thinking about data in terms of vectors:

- Each variable can be considered a vector
- Operations on a variable can be applied to all observations uniformly
- We can quickly reduce the number of variables for specific questions

No way do we want to write a `for` *loop for every little thing!*

Introducing Data Frames

In Python, the `pandas` library contains the necessary code to begin working with Data Frames. It is dependent on many functions in the `numpy` library.

```
import pandas as pd # Import the library for use
```

Creating a Data Frame

Create an empty Data Frame:

```
data = pd.DataFrame()
```

A Data Frame is a class that accepts the following parameters:

- `data`
- `index` (for referencing individual rows)
- `columns` (so you can name your variables)
- `dtype` (specify the **kind** of data for each column/variable)
- `copy` (whether or not the data should be duplicated in memory)

Creating a Data Frame

We can also use pandas to easily read many types of files, and use them to create Data Frames containing the information stored in the file:

```
# CSV
data = pd.read_csv(your_filename_here.csv)
# or Excel
data = pd.read_excel(your_filename_here.xlsx)
# or STATA
data = pd.read_stata(your_filename_here.dta)
# or SAS
data = pd.read_sas(your_filename_here.sas7bdat)
# or SQL
data = pd.DataFrame(engine.connect().execute(text(QUERY)))
# and many others!
```

Referencing a Single Column

To access a list of all of the column names in your Data Frame:

```
data.columns
```

To then access (slice) a single column:

```
data['Column_Name']
```

To slice several columns at once into a new Data Frame, pass a list of column names:

```
data[['Column1','Column2']]
```

Slicing the Data Frame - Hard Version

Two selection (or slicing) tools allow us to quickly subset our data.

```
data.iloc[row_selection, column_selection]
```

With the `.iloc` method, we can provide **integer**-based selections, or choose to select all rows or columns, and only subset on a single dimension.

```
data.iloc[:, 0] # Selects all rows, and first column
```

Slicing the Data Frame - Easy Version

Two selection (or slicing) tools allow us to quickly subset our data across both axes.

```
data.loc[row_selection, column_selection]
```

With the `.loc` method (now with no `i`), we can provide **name**-based selections, choose to select all rows or columns, and more.

```
data.loc[:, 'ColumnName'] # Selects all rows, one column
```

Slicing the Data Frame

With the `.loc` method (now with no `i`), we can also provide **logic**-based selections, creating subsets based on conditions.

```
data.loc[data['Column1'] == some_value, :]
# Selects only the observations (rows) where the
#   condition is met
```

Transforming our Data

We can quickly transform the data in a given column using the slicing techniques from above:

```
# Log the values of a variable, replace old column
data.loc[:,'Column1'] = np.log(data['Column1'])

# Difference two variables - make new column!
data['newColumn'] = data['Column1'] - data['Column2']
```

Because the variable doesn't exist yet, we don't use the .loc syntax here. Instead, we just create a new column by naming it! Super easy!

Transforming our Data

We can choose an index from among our columns, instead of the arbitrary ascending numbers assigned by default:

```
data.set_index('transaction_id')
```

Or, we can establish a multi-level index by passing a list of columns:

```
data.set_index(['year', 'month', 'day'])
```

Remember! Indices MUST be unique values! In the case of a multi-level index, the combination of values from the multiple columns must be unique for each row to be a valid index.

Datetimes and Date Processing

Processing Datetimes is also easy with built-in Pandas functionality:

```
data['myDate'] = pd.to_datetime(data['stringDateColumn'],
  format = '%Y%m%d', # Need to indicate the correct
  errors = 'ignore') #   format for your data!
```

We can also parse the data into separate columns afterward:

```
data['week'] = data['myDate'].dt.week
data['day'] = data['myDate'].dt.day
```

More Date Processing

A full list of the ways you can process dates is available at https://pandas.pydata.org/pandas-docs/stable/user_guide/timeseries.html#time-date-components.

Cleaning Data

There are many operations that are not reasonable to perform with missing data. Any numeric transformation will fail to provide useful output where missing values exist.

```
# Resolve missing values in ALL columns at once
data.fillna(0, inplace = True)
# fills ALL missing values, overwrites original data

# Resolve missing values in single column
data['Column'].fillna(method='pad') # fill values forward
# We can use method 'backfill' to use the NEXT value,
#   and fill backwards
```

Map and Apply

`.map()` enables us to apply a function to each value item-by-item in a single column

`.apply()` provides the same functionality column-by-column or row-by-row (use the `axis=1` argument to specify that you want to operate on one row at a time)

- Each row is the input to the funtion, rather than a single value from one column

Generating Summary Statistics

Using the `describe` funtion will create summary tables easily, and Pandas can even export them to csv for use in reports (this is true of ANY data frame in general, too!).

```
data.describe()
```

If we want the table presented similar to academic journal formats, we can add a few arguments:

```
data.describe().T[['count','mean','std','min','max']]
# We need to transpose the data using .T before
#   we can select the descriptive stats we want to keep
# Add a .to_csv('myfile.csv') to that line to save
```

Other summary functions

Count unique values in a field:

```
data['column_of_interest'].nunique()
```

Show unique values in a field:

```
data['column_of_interest'].unique()
```

Show dimensions of a Data Frame:

```
data.shape
```

Using SQL with Python (optional!)

In order to handle data on a large scale, we frequently rely on SQL databases. In this class, we can practice with MySQL.

Here is a link to analogous code for many other database types: http://docs.sqlalchemy.org/en/latest/core/engines.html

Install MySQL connectors with the following command:

```
pip install sqlalchemy mysql-connector-python
```

Using SQL with Python

The first thing we need to do is to establish a connection to our database:

```
from sqlalchemy import create_engine

engineStr = 'mysql+mysqlconnector://viewer:'
```

We are using `mysql` via the `mysqlconnector` module. Next, we provide our `username:password`, which in this case is "viewer," with no password, so we do not enter text after the colon.

Using SQL with Python

The first thing we need to do is to establish a connection to our database (will vary based on SQL flavor and server):

```
from sqlalchemy import create_engine

engineStr = 'mysql+mysqlconnector://*username*:*password*'
engineStr += '@35.202.92.40:3306'
```

We need to direct the connection to our server, which is hosted at `35.202.92.40`, and can be reached through port `3306`.

Using SQL with Python

The first thing we need to do is to establish a connection to our database:

```
from sqlalchemy import create_engine

# SQL flavor, user, password
engineStr = 'mysql+mysqlconnector://*username*:*password*'
engineStr += '@35.202.92.40:3306' # Server Address
engineStr += '/nfl' # Database Name

engine = create_engine(engineStr) # Start the Engine
```

Last, we just need to include the database that we wish to access on the server. In this case, we can use `NFL`

Retrieve SQL Data with Pandas

Our next step is to write a `SELECT` statement using SQL, and then to pass it to Pandas for retrieval.

```
select = """SELECT * FROM game WHERE seas=2019"""

data = pd.DataFrame(engine.connect().execute(text(select)))
```

Want to learn a bit about SQL queries?

Feel free to take a look at some slides about writing SQL query code:

https://goo.gl/Lq2yC5

DuckDB, Pandas and Data Cleaning

We can actually use SQL to clean our data within Pandas by making use of the `duckdb` library.

Get started by using the following code:

```
import pandas as pd
import duckdb
```

If it isn't installed, you can install the library by running

```
!pip install duckdb # "!" only needed in notebooks
```

DuckDB, Pandas and Data Cleaning

```
edited_data = duckdb.sql(query_with_data_frame_as_table)
```

Using the DuckDB SQL syntax, we can then clean any dataset using the same tools that we would to extract data from a database!

We can aggregate, create new columns, group, and join across datasets, just like we would with SQL!

Lab Time!